Assignment 2 Response

Course: CSCI-UA.480-062 Computer Networking
Instructor: Jinkun Geng
Student: Omer Hortig
NetID: oh2065
Date: March 2, 2026

2. Moving Averages and Retransmission Timeouts

2.1 Overview of the Code

Why do we need a minimum value of the retransmission timeout?

A minimum retransmission timeout is necessary to prevent the sender from retransmitting packets too aggressively. Because RTT calculations fluctuate with normal network jitter, the EWMA estimate may sometimes be very small. If the timeout were allowed to shrink too much, the sender could then trigger unnecessary retransmissions before acknowledgments have a fair chance to arrive, leading to duplicate packets.

Why do we need a maximum value of the retransmission timeout?

A maximum retransmission timeout is needed to prevent the sender from waiting too long before retransmitting a lost packet. If RTT estimates spike due to some temporary issue, the excessively large calculated timeout would cause the protocol to stall for a long time before retrying, significantly diminishing throughput and responsiveness.

2.3 Timeout Simulation

short-spike:

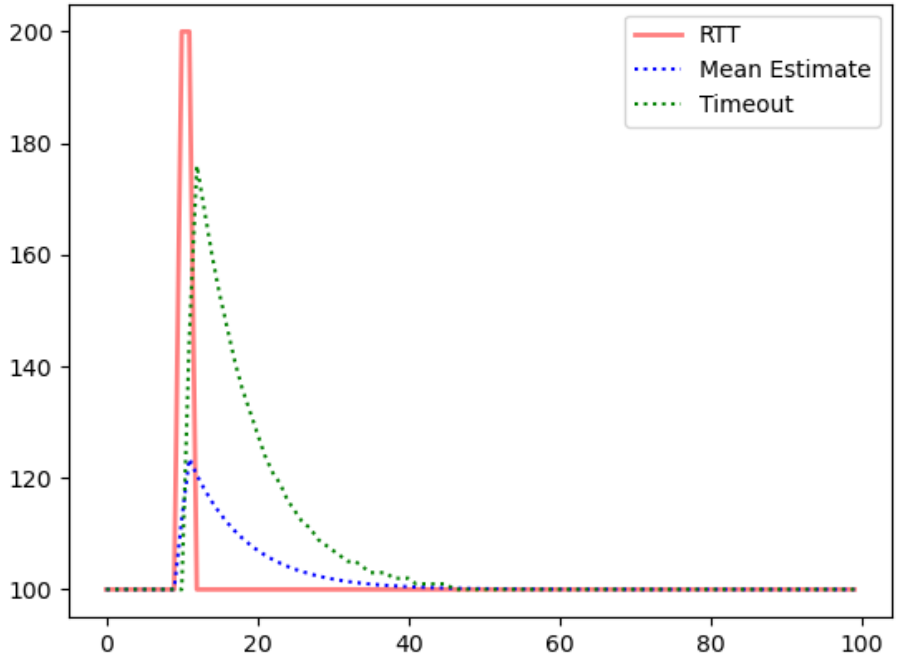

Figure 1: Timeout simulation — short spike scenario

long-spike:

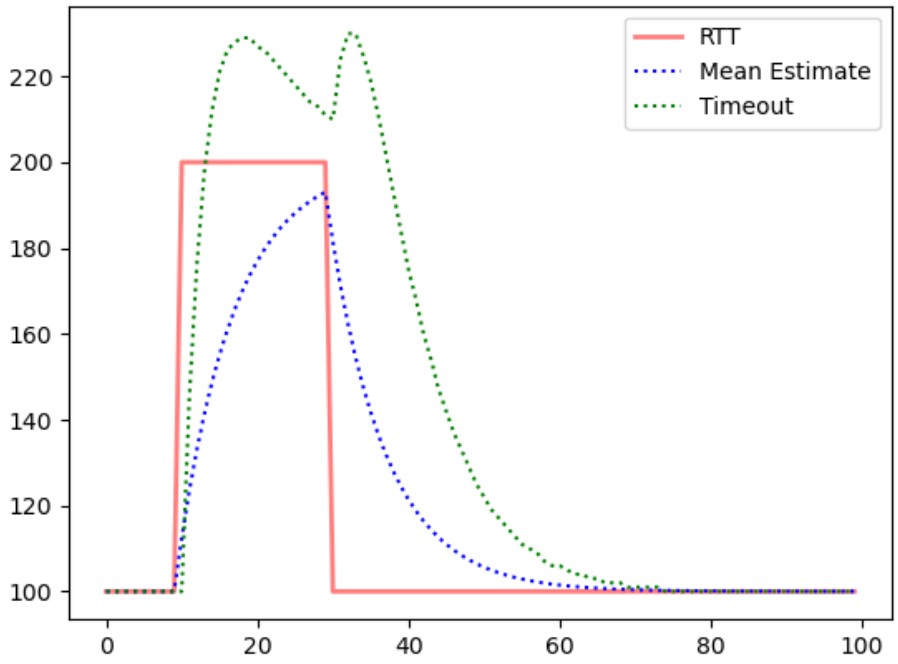

Figure 2: Timeout simulation — long spike scenario

permanent-change:

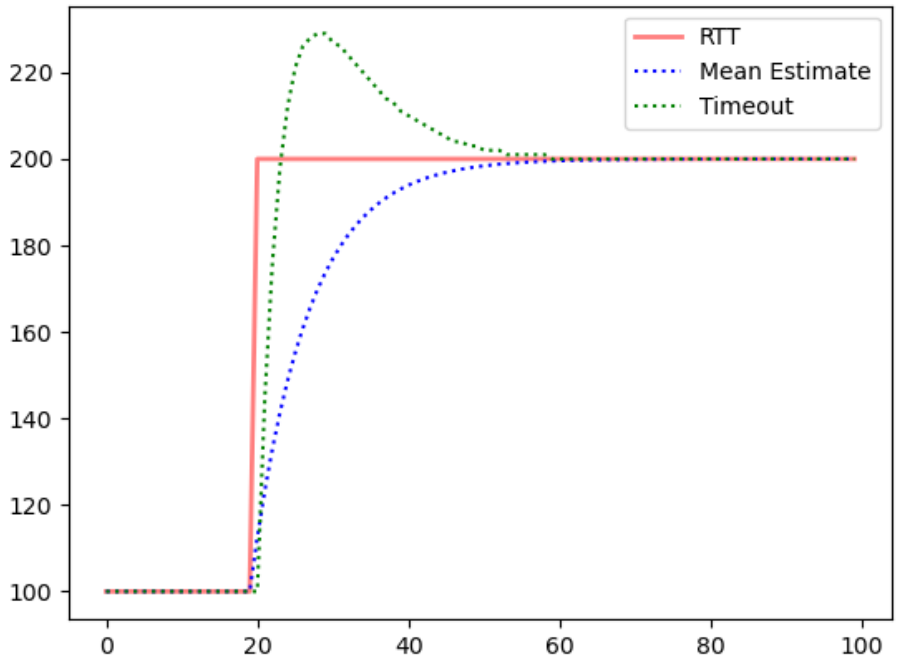

Figure 3: Timeout simulation — permanent change scenario

high-variance:

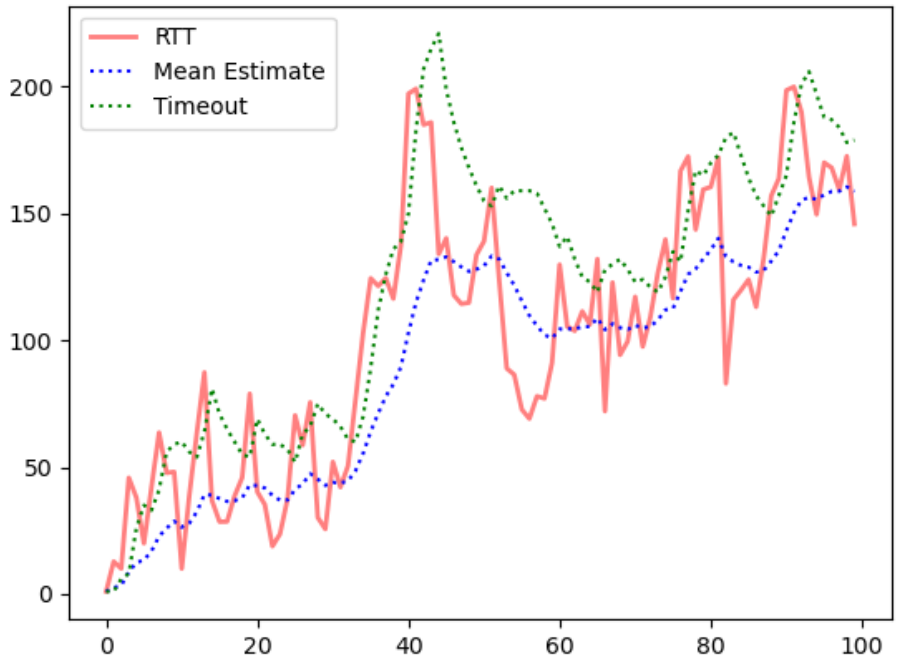

Figure 4: Timeout simulation — high variance scenario

3. Reliability

3.3 Stop-and-Wait Protocol

To evaluate the Stop-and-Wait protocol, throughput was simulated across a range of minimum RTT values (10, 20, 40, 50, 80, 100, 150, 200, 300, and 500 ms). The results are plotted below as throughput $\frac{W}{RTT}$ versus $\frac{1}{RTT_{\min}}$:

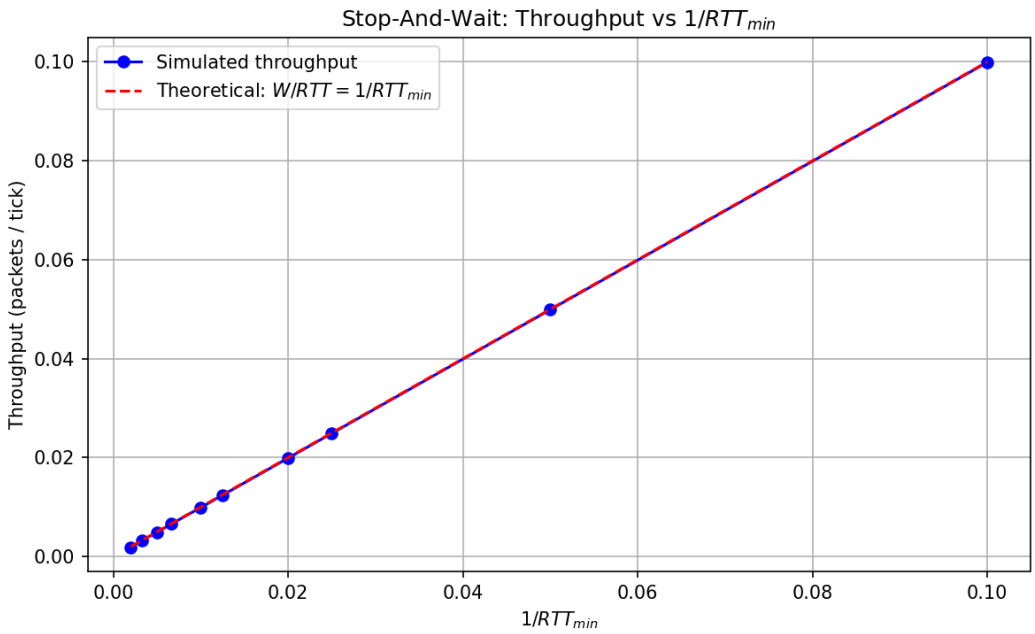

Figure 5: Stop-and-Wait throughput vs. $\frac{1}{RTT_{\min}}$

The blue simulated dots lie almost exactly on the red theoretical dashed line ($\frac{W}{RTT} = \frac{1}{RTT_{min}}$) across the entire range – a clean linear relationship confirming that Stop-And-Wait achieves exactly 1 packet per RTT, as expected from the $\frac{W}{RTT}$ model with $W = 1$.

After setting `--loss-ratio=0.01` to simulate a 1% packet loss, the throughput graph is as follows:

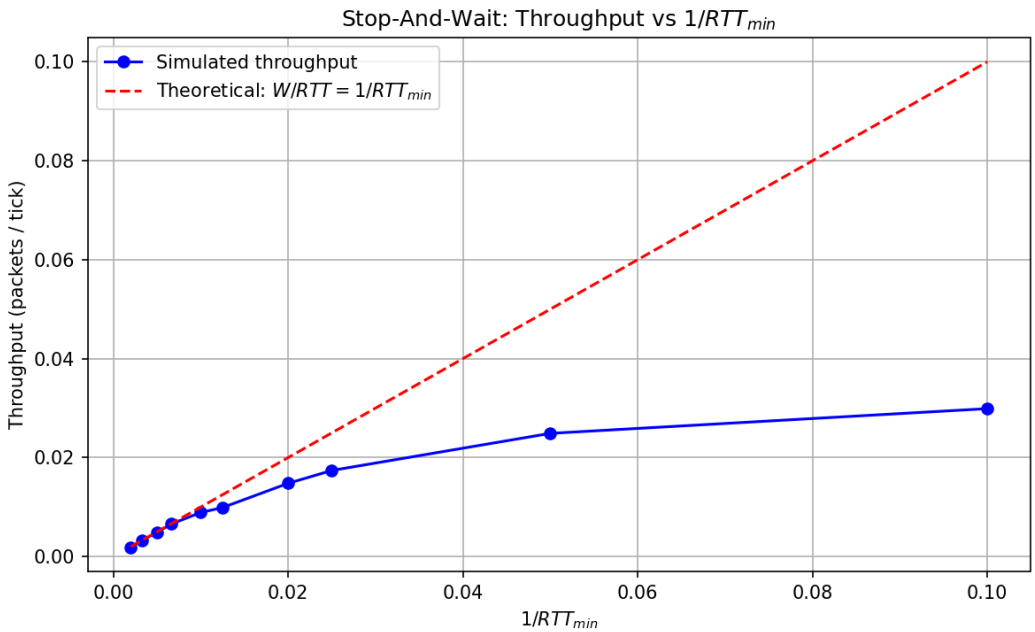

Figure 6: Stop-and-Wait throughput vs. $\frac{1}{RTT_{\min}}$ with 1% loss

Now, the simulated throughput falls below the ideal $\frac{W}{RTT} = \frac{1}{RTT_{min}}$ line because each dropped packet forces the host to wait 1000 ticks (the minimum timeout) before retransmitting — far longer than the actual RTT. This penalty is proportionally worse at small RTT_{min} values: for example, at $RTT_{min} = 10$, the host wastes 100 RTTs per drop, whereas at $RTT_{min} = 500$ it only wastes 2 RTTs per drop, so the throughput curve diverges more from the theoretical line at higher $\frac{1}{RTT_{min}}$ (smaller RTTs).

3.4 Sliding Window Protocol

The plot below compares the simulated sliding window throughput (solid lines) against the theoretical prediction $\frac{W}{RTT_{min}}$ (dashed lines) for window sizes W = 1, 2, 5, 10, 20, with no packet loss:

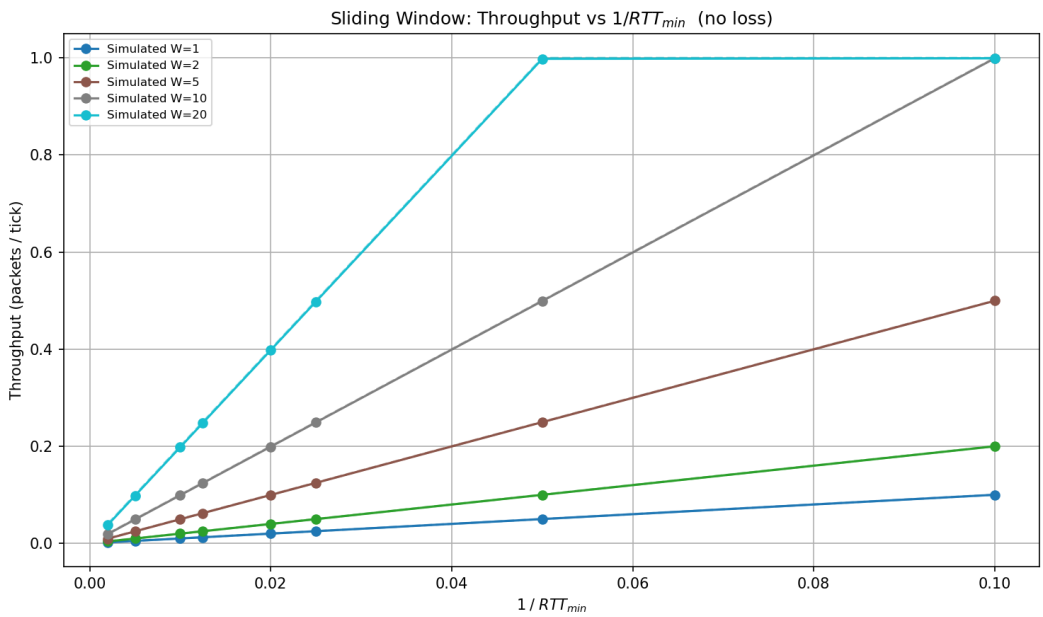

Figure 7: Sliding Window throughput vs. $\frac{1}{RTT_{\min}}$ (no loss)

The simulated throughput matches the theoretical prediction exactly – covering the dashed lines – across all window sizes and RTT_{min} values. For smaller windows (W = 1, 2, 5), throughput scales linearly with $\frac{W}{RTT_{min}}$ as expected. For larger windows ($W = 10$ at $RTT_{min} = 10$, $W = 20$ at $RTT_{min} \leq 20$), throughput correctly caps at 1.0 packet/tick since $\frac{W}{RTT_{min}}$ exceeds the link capacity.

The plot below repeats the same comparison but introduces a 1% independent packet loss rate, showing how loss degrades throughput relative to the theoretical $\frac{W}{RTT_{min}}$:

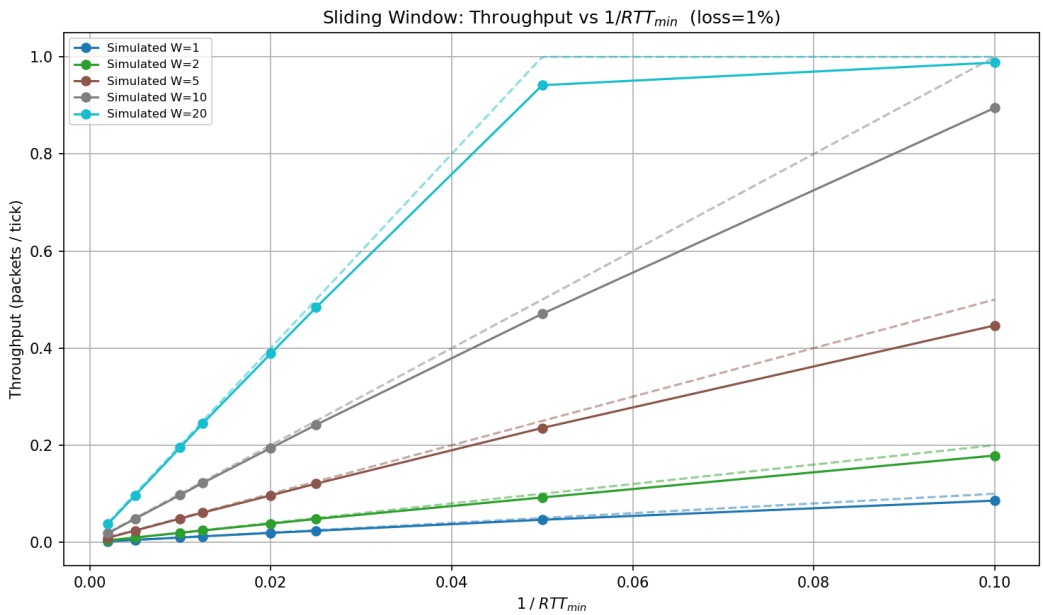

Figure 8: Sliding Window throughput vs. $\frac{1}{RTT_{\min}}$ with 1% loss

Under 1% loss, the protocol continues to function correctly — all packets are eventually delivered and throughput still scales with $\frac{1}{RTT_{\min}}$. However, simulated throughput now consistently falls below the theoretical dashed lines. The gap is most pronounced at high $\frac{1}{RTT_{\min}}$ (small RTT_{min}) where the pipeline is nearly full: lost packets trigger retransmissions that compete with new transmissions for link capacity, and the in-order delivery counter stalls until the lost packet is successfully retransmitted. At lower $\frac{1}{RTT_{\min}}$ (large RTT_{min}), the pipeline is under-utilized, so sporadic retransmissions have minimal impact.

This heatmap shows the percentage divergence between simulated and predicted throughput as a function of both window size (y-axis) and RTT_{min} (x-axis) under 1% loss:

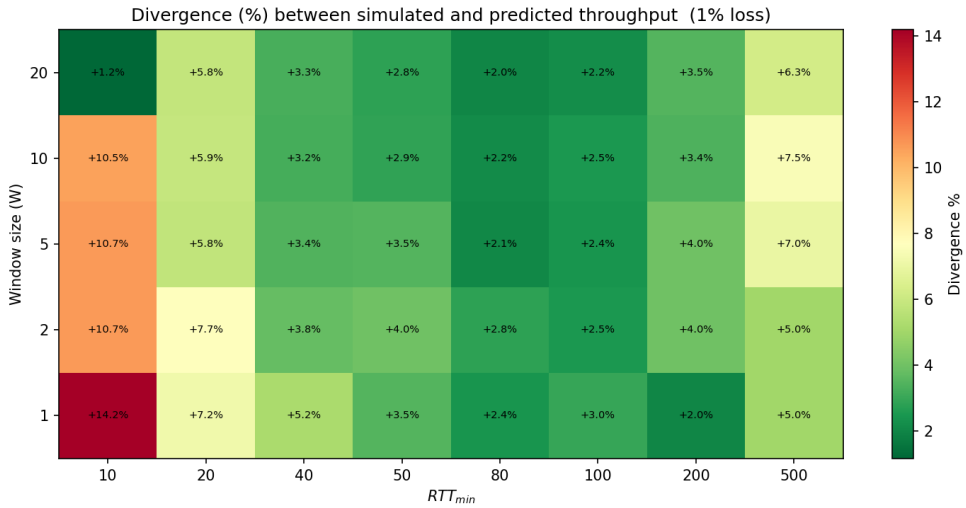

Figure 9: Throughput divergence heatmap (1% loss)

The divergence pattern reveals two key trends. First, at small RTT_{min} (left column), divergence peaks at 10–14% for smaller windows ($W = 1$–10) because the sender is transmitting at or near link capacity, so every retransmission directly displaces a new packet. The notable exception is $W = 20$ at $RTT_{min} = 10$, which shows only ∼1.2% divergence: here $\frac{W}{RTT_{min}} = 2$ far exceeds the link capacity of 1, so the excess window provides spare capacity to absorb retransmissions without displacing new data. Second, at large RTT_{min} (right side), divergence rises again to 5–7.5% because the total number of completed round trips is small, making each lost packet a proportionally larger fraction of delivered packets. The minimum divergence (∼2%) occurs at mid-range RTT_{min} values (80–100), where the pipeline is moderately utilized and retransmissions are infrequent enough to absorb without significant throughput impact.

Finally, the event plot below shows the exact ticks at which original transmissions (blue) and retransmissions (red) occur during a 2000-tick simulation with $W = 20$, $RTT_{min} = 10$, and 1% loss — options where $\frac{W}{RTT_{min}} = 2$ exceeds the link capacity:

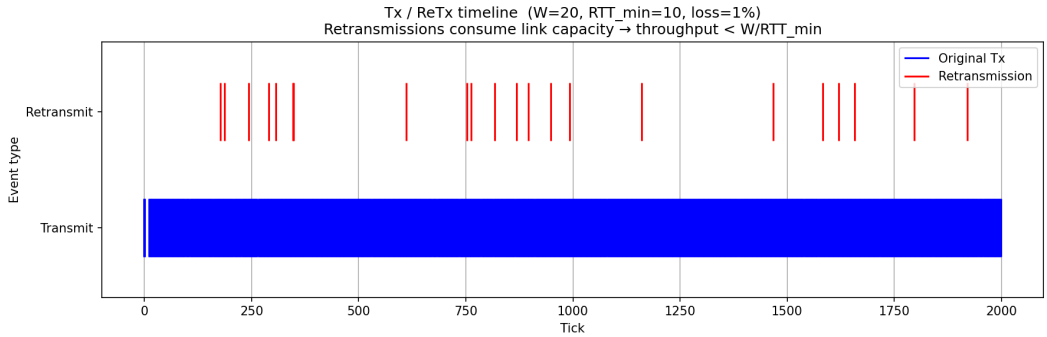

Figure 10: Transmission events (blue = original, red = retransmission)

The blue transmit band is continuous, showing that the sender keeps the pipeline full throughout. Red retransmission events are scattered across the timeline, each one representing a tick where the link carries a retransmitted packet instead of a new one. Since the link can only transmit 1 packet/tick and the sender is already saturating it, every retransmission directly reduces the effective throughput for new data. This explains why throughput falls below $\frac{W}{RTT_{min}}$: the retransmissions consume link capacity that would otherwise carry new packets, and stall the in-order sequence counter until the gap is filled.

4. AIMD

4.1 AIMD Protocol

The following window-size plots were generated for queue limits 1, 10, 20, and 50:

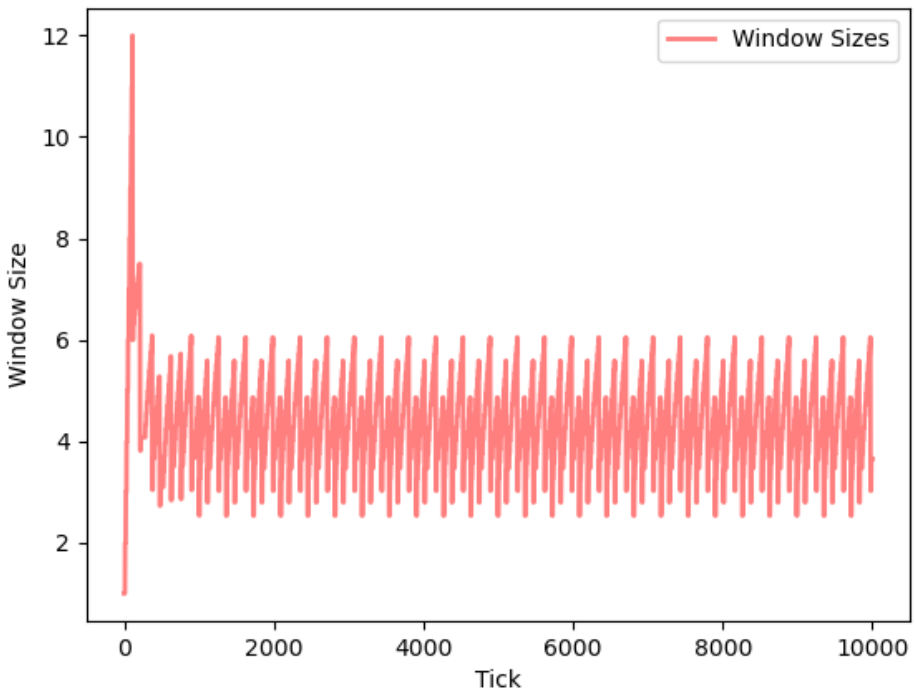

Figure 11: AIMD window size over time (queue limit = 1)

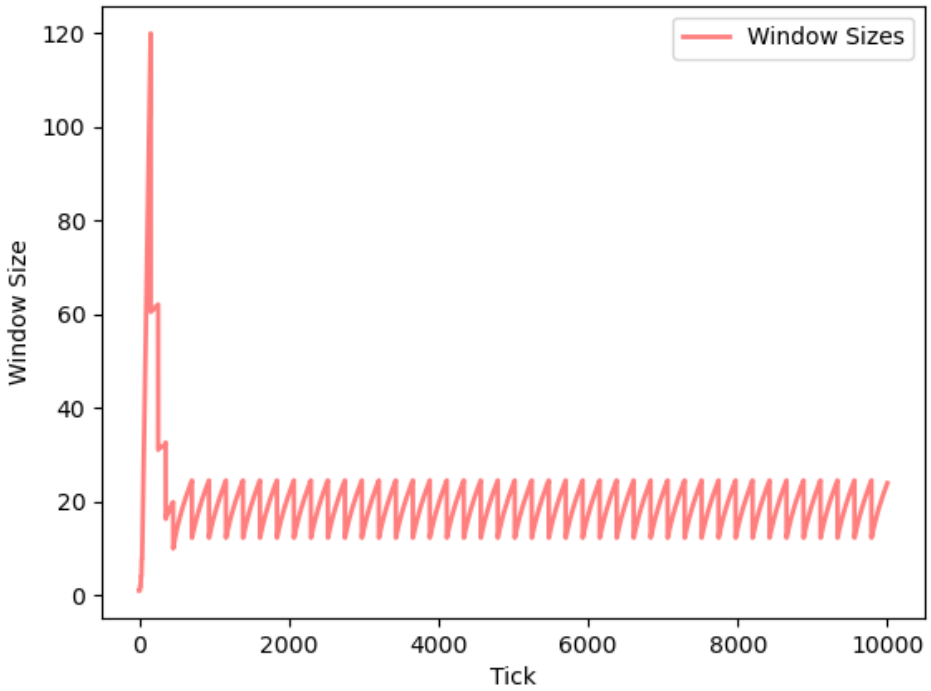

Figure 12: AIMD window size over time (queue limit = 10)

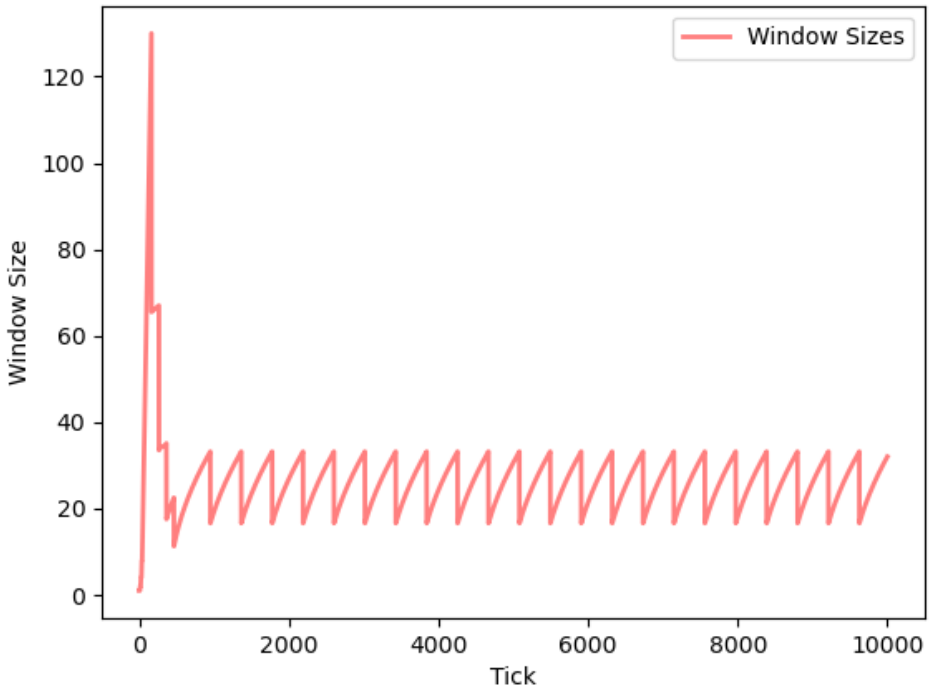

Figure 13: AIMD window size over time (queue limit = 20)

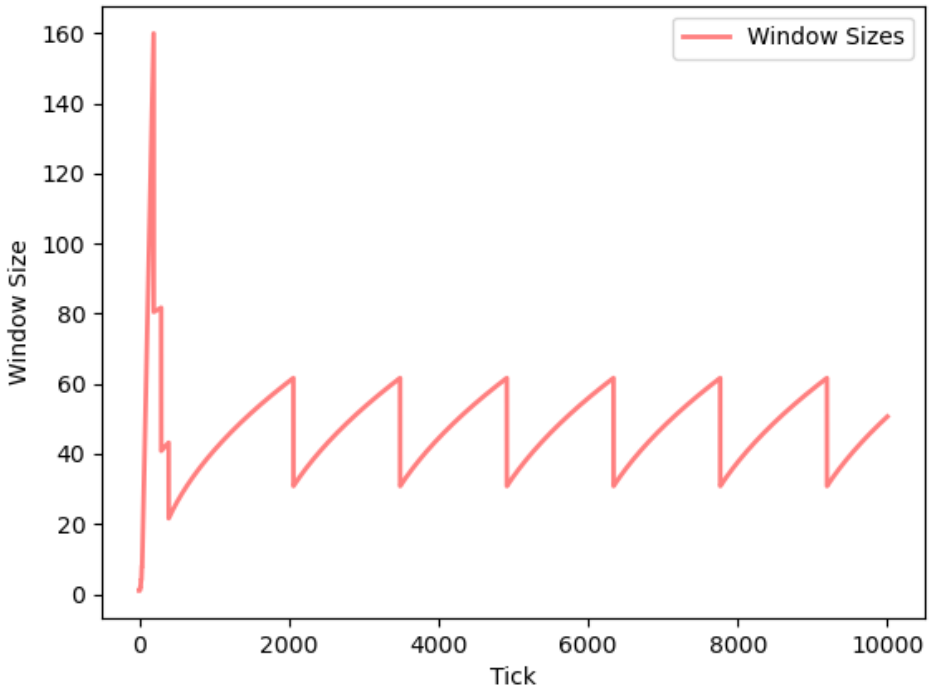

Figure 14: AIMD window size over time (queue limit = 50)

1) Additive increase, multiplicative decrease, and sawtooth period

Yes – all four plots show the AIMD behavior: a gradual linear increase in window size (additive increase), followed by a sharp drop after loss (multiplicative decrease), producing a sawtooth pattern.

Rough sawtooth periods measured from the window plots:

- Queue limit 1: about 350–400 ticks per cycle.
- Queue limit 10: about 430–450 ticks per cycle.
- Queue limit 20: about 470–500 ticks per cycle.
- Queue limit 50: about 1400–1500 ticks per cycle.

As queue capacity increases, the sender can grow to a larger window before overflow/loss, so each sawtooth cycle becomes longer.

2) Throughput of AIMD

Using throughput $\approx \frac{\text{max in-order received sequence}}{\text{simulation ticks}}$ and a 10,000-tick run:

Queue limit	Max in-order sequence	Throughput (packets/tick)
1	929	$\frac{929}{10000} = 0.0929$
10	9770	$\frac{9770}{10000} = 0.9770$
20	9885	$\frac{9885}{10000} = 0.9885$
50	9964	$\frac{9964}{10000} = 0.9964$

Interpretation: queue limit 1 severely underutilizes the link, while queue limits 10, 20, and 50 achieve throughput very close to link capacity (1 packet/tick).

3) Purpose of queue capacity in AIMD

The bottleneck queue absorbs short-term mismatches between sender rate and link service rate. In AIMD, the congestion window increases over time, so temporary bursts naturally occur before a loss event cuts the window. Without enough queue space, packets are dropped too early and too often, forcing frequent multiplicative decreases and keeping the average window too small (low throughput). With sufficient queue capacity, the sender can keep the pipeline full between loss events, so average throughput stays close to the link capacity.